

資料探勘hw3

一、資料前處理

以下為資料的欄位

Feature	Description
ACTION	是否授權, 1 代表核准, 0 代表未核准
RESOURCE	資源識別 ID
MGR_ID	員工主管 ID
ROLE_ROLLUP_1	公司角色群組分類 ID 1
ROLE_ROLLUP_2	公司角色群組分類 ID 2
ROLE_DEPTNAME	公司角色所屬部門 ID
ROLE_TITLE	公司職稱 ID
ROLE_FAMILY_DESC	角色家族延伸描述 ID
ROLE_FAMILY	角色家族 ID
ROLE_CODE	角色代碼 ID

本作業使用的主要輸入特徵為 RESOURCE、MGR_ID、ROLE_ROLLUP_1、ROLE_ROLLUP_2、ROLE_DEPTNAME、ROLE_TITLE、ROLE_FAMILY_DESC、ROLE_FAMILY 與 ROLE_CODE。

(一) target encoding

在多數繳交的版本中，資料前處理大部分是使用target encoding，這些欄位會先經過out-of-fold target encoding。Target encoding 的概念是以某個類別在訓練資料中對應的 target 平均值作為新的數值特徵。例如，若某個資源代碼在歷史資料中較常出現在授權案例中，則其 target encoded value 會較高。

然而若直接使用整份 training data 的 ACTION 對同一份 training data 做 target encoding，某些出現次數很少的類別可能會把自己的答案洩漏進特徵中。為降低此問題，本作業採用 OOF target encoding。訓練資料會先以 `StratifiedKfold` 切成多份；在編碼某一 fold 時，只使用其他 folds 的資料建立 encoding mapping，再將結果填回該 fold。測試資料則使用完整訓練資料建立 mapping 後進行編碼。

此外target encoding 中還加入 smoothing, 以減少稀有類別造成的過度擬合, 當某類別出現次數很少時, 其 encoded value 會更接近整體平均值, 當該類別累積足夠多樣本後, 模型才會更相信該類別本身的 target 平均值, 以下為程式碼。

```
def make_oof_target_encoding(train, test, features, n_splits=5, smoothing=10, seed=42):
    y = train[TARGET].values
    global_mean = train[TARGET].mean()
    train_encoded = pd.DataFrame(index=train.index)
    test_encoded = pd.DataFrame(index=test.index)
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)

    for feature in features:
        encoded_col = f"{feature}_te"
        train_encoded[encoded_col] = np.nan

        for train_idx, valid_idx in skf.split(train, y):
            fold_train = train.iloc[train_idx]
            fold_valid = train.iloc[valid_idx]
            mapping = smooth_target_mean(
                fold_train[feature],
                fold_train[TARGET],
                global_mean=fold_train[TARGET].mean(),
                smoothing=smoothing,
            )
            train_encoded.loc[fold_valid.index, encoded_col] = (
                fold_valid[feature].map(mapping).fillna(global_mean)
            )

        full_mapping = smooth_target_mean(
            train[feature],
            train[TARGET],
            global_mean=global_mean,
            smoothing=smoothing,
        )
        test_encoded[encoded_col] = test[feature].map(full_mapping).fillna(global_mean)

    return train_encoded, test_encoded
```

(二) one hot encoding

除了 target encoding, 在些許繳交的版本, 也有對使用One-hot encoding進行資料前處理, 以下為程式碼。

```
model = make_pipeline(
    OneHotEncoder(handle_unknown="ignore"),
    LogisticRegression(max_iter=2000, class_weight="balanced", random_state=args.seed),
)
```

二、模型訓練流程

本作業的模型訓練流程可概括為以下步驟。首先讀取訓練資料與測試資料，接著選取九個 categorical features 作為輸入欄位。不同版本會依實驗目的使用 target encoding 或 one-hot encoding 將類別資料轉為模型可使用的格式。完成前處理後，模型會先透過 5-fold stratified cross-validation 評估 AUC，再選擇不同的模型完整訓練資料重新訓練，最後對 test data 輸出 ACTION = 1 的機率並產生 Kaggle submission 檔案

三、模型改進流程

本研究主要採用三種模型，分別是logistical regression、random trees、gradient_boosting，並分別搭配不同的訓練參數，共繳交六個不同的版本

a. target encoding + logistic regression

第一個版本v1以target encoding + Logistic Regression去做為訓練，想看這種簡單的模型能夠達到怎麼樣的表現，以下為Logistic Regression參數設定。

Logistic Regression參數表

參數	設定值
max_iter	2000
class_weight	balanced

b. one hot encoding + logistic regression

接著想去比較 categorical encoding 的差異是否能夠提升表現，第二個版本v1.1 保留 Logistic Regression，僅將 target encoding 改成 one-hot encoding，Logistic Regression的參數保持與版本v1一樣。此實驗能較單純地觀察特徵表示方式對同一模型的影響。

c. target encoding + random forest

在完成線性模型實驗後，第三個版本v2改用target encoding + Random Forest作為實驗，由於員工權限判斷可能不只依賴單一欄位，而與角色、主管、部門及資源之間的組合關係有關，因此使用樹模型測試其捕捉非線性關係的能力，以下為Random Forest參數設定。

Random Forest

參數	設定值
n_estimators	500
max_depth	12
min_samples_leaf	10
class_weight	balanced_subsample
n_jobs	-1

d. target encoding + random forest (fine tune)

在第四個版本v2.1 中，模型仍使用 Random Forest，但調整模型容量。樹數由 500 增加到 800，最大深度由 12 增加到 16，最小 leaf 樣本數由 10 降低到 5。此修改希望讓模型學到更細緻的 target-encoded feature 關係，同時利用更多 trees 降低預測波動，以下為Random Forest參數設定。

Random Forest

參數	設定值
n_estimators	800
max_depth	16
min_samples_leaf	5
class_weight	balanced_subsample
n_jobs	-1

e. target encoding + gradient boosting

在完成 Random Forest 實驗後，第五個版本 v3 改用target encoding + Gradient Boosting 作為另一種 tree ensemble 方法。與 Random Forest 透過多棵樹平行平均不同，Gradient Boosting 會逐步加入弱樹模型修正前一階段的殘差錯誤，因此對資料中複雜的特徵交互關係可能有更強的擬合能力。此版本以較保守的學習率與較淺的樹深度作為起點，以下為Gradient Boosting參數設定。

Gradient Boosting

參數	設定值
n_estimators	300
learning_rate	0.05
max_depth	3
min_samples_leaf	20
subsample	0.8

f. target encoding + gradient boosting (fine tune)

第六個版本 v3.1 維持 Gradient Boosting 架構，針對學習動態與模型容量進行調整。將學習率由 0.05 降低至 0.03，同時增加 boosting 階段數至 500，希望透過更小步伐但更多次的修正來提升最終精度。此外將 min_samples_leaf 由 20 降低至 10，放寬葉節點限制，測試 v3 是否因過度限制樹的生長而未能充分學習資料中的細粒度模式，以下為Gradient Boosting參數設定。

Gradient Boosting

參數	設定值
n_estimators	500
learning_rate	0.03
max_depth	3
min_samples_leaf	10
subsample	0.8

四、模型評估

本作業使用 Kaggle leaderboard score 共同評估模型表現，以下為各版本的表現。

a. target encoding + logistic regression

	submission_1_target_encoding_logistic.csv Complete (after deadline) · 21h ago	0.86635	0.87230	☐
---	---	----------------	----------------	--------------------------

b. one hot encoding + logistic regression

	submission_1_1_one_hot_logistic.csv Complete (after deadline) · 9h ago	0.88165	0.88803	☐
---	--	----------------	----------------	--------------------------

c. target encoding + random forest

	submission_2_target_encoding_random_fores... Complete (after deadline) · 21h ago	0.86978	0.87703	☐
---	--	----------------	----------------	--------------------------

d. target encoding + random forest (fine tune)

	submission_2_1_target_encoding_random_for... Complete (after deadline) · 9h ago	0.86755	0.87416	☐
--	---	----------------	----------------	--------------------------

e. target encoding + gradient boosting

	submission_3_target_encoding_gradient_boos... Complete (after deadline) · 21h ago	0.87009	0.87542	☐
---	---	----------------	----------------	--------------------------

f. target encoding + gradient boosting (fine tune)

	submission_3_1_tuned_gradient_boosting.csv Complete (after deadline) · 9h ago	0.87069	0.87545	☐
---	---	----------------	----------------	--------------------------